

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
```

```
file = "/content/communication_system_fault__dataset.csv"

#Read the file as a single column, assuming each line is a single string
temp_data = pd.read_csv(file, header=None)

#The first row contains the header string
header_line = temp_data.iloc[0, 0]
column_names = [col.strip() for col in header_line.split(',')]

#The rest of the rows contain the data
data_lines = temp_data.iloc[1:, 0]

#Split each data line into multiple columns
processed_data = data_lines.str.split(',', expand=True)
processed_data.columns = column_names

#Convert columns to appropriate types
numeric_cols = ['Signal_Strength', 'SNR', 'Packet_Loss', 'Latency', 'Temperature',
for col in numeric_cols:
    processed_data[col] = pd.to_numeric(processed_data[col], errors='coerce')

#Assign the processed data back to 'data' variable
data = processed_data.copy()
```

```
print(data.head())
```

	Signal_Strength	SNR	Packet_Loss	Latency	Temperature	Power_Level	\
1	-45	35	1	20	35	90	
2	-50	32	2	25	36	88	
3	-55	30	3	30	37	85	
4	-60	28	5	40	40	80	
5	-65	25	8	55	42	75	

	Fault_Status
1	Normal
2	Normal
3	Normal
4	Normal
5	Fault

```
print(data.tail())
```

	Signal_Strength	SNR	Packet_Loss	Latency	Temperature	Power_Level	\
16	-82	12	30	140	55	55	
17	-46	36	1	18	33	95	
18	-52	33	2	28	35	89	
19	-67	24	10	65	44	74	
20	-77	14	27	130	52	58	

	Fault_Status
16	Fault
17	Normal
18	Normal
19	Fault
20	Fault

```
print(data.isnull().sum())
```

```
Signal_Strength    0
SNR                0
Packet_Loss        0
Latency            0
Temperature        0
Power_Level        0
Fault_Status       0
dtype: int64
```

```
from sklearn.preprocessing import LabelEncoder
```

```
label_encoder = LabelEncoder()
data['Fault_status_Encoded'] = label_encoder.fit_transform(data['Fault_Status'])
```

```
X = data.drop(['Fault_Status', 'Fault_status_Encoded'], axis=1)
y = data['Fault_status_Encoded']
```

```
X = pd.get_dummies(X, drop_first=True)
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)
```

```
#Train-test split
#-----

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
#Feature scaling
#-----

scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
# Train Logistic Regression Model
#-----

model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)
```

▼ **LogisticRegression** ⓘ ?

```
LogisticRegression(max_iter=1000)
```

```
#Prediction
#-----

y_pred = model.predict(X_test)
```

```
#Evaluation
#-----

print("\nAccuracy:")
print(accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Accuracy:
1.0

Confusion Matrix:
[[1 0]
 [0 3]]

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1
1	1.00	1.00	1.00	3
accuracy			1.00	4
macro avg	1.00	1.00	1.00	4
weighted avg	1.00	1.00	1.00	4

